

Real-world (make it feel alive with streaming) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `streaming_practice.py`: a normal call waits for the entire reply before showing you anything — for a long answer, that's several seconds of silence. Streaming prints each piece as it arrives, which is the difference between a chatbot that feels instant and one that feels stuck. **If not:** the Build Task's `stream_message()` would be the first time you ever saw a chunk-by-chunk response, with no smaller version to check your understanding against.

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — the direct way

```
# streaming_practice.py
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Explain a REST API in 3 sentences."},
]

stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    stream=True,
)

for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")

print()
```

Expected output: the 3-sentence explanation appearing on screen piece by piece, noticeably faster to *start* than the same call without `stream=True`.

This works and shows the streaming effect clearly. It doesn't keep track of the full reply text anywhere, so if you needed to save it (into a conversation history list, for example) you'd have nothing to append.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `stream_reply()` returns the accumulated text

```
# streaming_practice.py
def stream_reply(history: list) -> str:
    # why: returns the full text once streaming finishes, so this can plug
    # straight into the same history list the memory exercise already builds.
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    for chunk in stream:
        # how: each chunk carries one small piece of the reply in
        # .delta.content – None on chunks that carry no new text.
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply = full_reply + piece

    print()
    return full_reply

def main() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain a REST API in 3 sentences."},
    ]
    reply = stream_reply(history)
    history.append({"role": "assistant", "content": reply})

if __name__ == "__main__":
    main()
```

Expected output: identical streamed text to Basic, plus `history` now correctly ends with the assistant's full reply appended.

Difference from Basic: `stream_reply(history)` returns the full accumulated text once streaming finishes, so it can plug directly into the conversation-memory loop from the previous exercise — call it instead of `get_reply(history)`, append its return value to `history`, and you have a chatbot with both real memory and streamed output. Full type hints document that it takes a history list and hands back the finished reply as a string, hiding the chunk-by-chunk detail from the caller.

Which one should you actually write? Intermediate's version is what this document's Build Task uses. If you ever turn on `stream_options={"include_usage": True}` for token counts on a streamed call, guard with `if not chunk.choices: continue` before indexing — the final usage-only chunk carries an empty `choices` list, and indexing into it crashes with `IndexError` on the very last chunk of an otherwise-successful reply. This document's Build Task doesn't need usage tracking on the stream, so it doesn't need the guard either — but know it's there for the day you add one.